Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Sensör kavramı
  - Sensörler, cihazların (akıllı telefon, akıllı saat) gözü kulağı olarak kabul edilebilir.
  - Mevcut sensörler insanlardan daha hassas ölçümler yapabilir (düşme tespiti, adım sayma, sıcaklık ölçümü).
  - Cihazlardaki sensörler küçük boyutlarına rağmen yeterli olsa da, profesyonel (plasman) sensörler kadar detaylı ölçüm yapamayabilir.
- Sensör çeşitliliği
  - Farklı cihazlarda farklı sensörler bulunabilir; program yazarken bu durumun gözetilmesi önemlidir.
  - Her sensörün yetenekleri farklıdır: ölçebileceği aralık (range), çözünürlük (resolution), ihtiyaç duyduğu güç ve enerji.
  - Çok sık arka planda değer okunan sensörler için bu özelliklerin gözetilmesi kritik öneme sahiptir.
- Sensör türleri
  - Hareket sensörleri: ivme ölçer (accelerometer), jiroskop (gyroscope), yerçekimi sensörü.
  - Pozisyon sensörleri: manyetometre, proximity (yakınlık) sensörü.
  - Çevresel sensörler: ışık, sıcaklık, basınç, nem sensörü.
  - Donanımsal (hardware) ve yazılımsal (software) sensörler ayrımı; yazılımsal sensörler donanım sensörlerinden elde edilen değerler kullanılarak hesaplanır (örneğin yerçekimi sensörü, doğrusal ivme ölçer).
- Sensör framework'ı
  - Dört temel bileşen:
    1. SensorManager: sensörlere erişim ve sisteme entegrasyon/çıkarma sorumluluğu.
    2. Sensor sınıfı: sensörün yeteneklerini paylaşır.
    3. SensorEvent: sensörden veri üretildiğinde oluşan obje; verinin ne zaman üretildiği, hareketin değeri, hangi sensörden geldiği bilgilerini saklar.
    4. SensorEventListener: tetikleme sırasında değişen değerleri takip etmek için listener.
- Sensör kullanım adımları
  - SensorManager objesi oluşturulur (getSystemService ile sensör servisi alınır).
  - getSensorList ile cihazdaki sensörler listelenir (type all veya type accelerometer gibi).
  - Cihazda birden fazla aynı tip sensör olabilir (hastal sebepler, enerji verimliliği, yedeklilik).
  - Sensör özellikleri: getResolution, getMaximumRange, getPower, getVendor, getVersion.
  - Sensör kaydı: registerListener ile sensör dinlenmeye başlanır, unregister ile durdurulur.
  - Sensör sorgulama: queryIntentActivities benzeri bir kontrol ile ihtiyaç duyulan sensör var mı yok mu tespit edilir.
- Sensör frekansı (veri toplama hızı)
  - Saniyedeki veri toplama hızına "frekans" denir (örneğin 50 Hz = saniyede 50 veri, 20 ms'de bir).
  - Akselerometre ve jiroskop gibi hareket sensörleri saniyede 200 defaya kadar veri

toplayabilir.
  - Işık sensörü gibi sensörler daha düşük aralıklarla (saniyede 1-2) değer üretir; sık okuma anlamsızdır.
  - Hangi sensörle çalışılıyorsa çözünürlük ve frekans set edilebilir.
- SensorEventListener metotları
  - onAccuracyChanged: sensör hassasiyeti değiştiğinde çağrılır.
  - onSensorChanged: her veri değişiminde çağrılır; her bir değişim gerçekleştiğinde sensör event üzerinden değerler alınır.
  - Birden fazla sensör okunuyorsa hangi sensörden değer geldiği kontrol edilmelidir.
  - Sürekli veri alan sensörler (accelerometer, gyroscope) "continuous" olarak event üretir.
  - Step counter gibi sensörler sadece değişim olduğunda event üretir ("on-change").
  - Significant motion sensör gibi trigger sensörler sadece belirli bir olay tetiklendiğinde bir kez event üretir ("one-shot").
- Sensör veri formatı
  - 3 eksenli sensörler (X, Y, Z) için values[0], values[1], values[2].
  - 5 eksenli (kalibre olmamış) sensörler için values[0..4].
  - Tek eksenli sensörler (örn. ışık) için sadece values[0].
  - Kalibre ve kalibre olmayan sensörlerin farklı veri formatları.
- Sensör kayıt ve kaldırma (register/unregister)
  - Kullanılmayan sensörlerin mutlaka unregister edilmesi gerekir (pil ömrü için).
  - Activity pause olduğunda (örneğin oyun duraklatıldığında) sensör unregister edilebilir.
  - Arka planda çalışan servisler için sensör kaydı gerekli olduğunda, gerekli durumlarda unbind edilerek durdurulabilir.
  - Manifest'te Google Play Filter ile sensör/kamera gibi parçalar required="true" olarak tanımlanabilir; olmayan cihazlarda uygulama Google Play'de görünmez.
  - required="false" yapılırsa iki farklı yol (sensörü olan/olmayan) kodlanabilir.
- Android Run-time izin mekanizması ve sensörler
  - Sensörlere erişim için de izin sistemi kullanılabilir.
  - "Allow once, allow while using the app" gibi seçenekler sunulabilir.
  - Android 9 ile birlikte sensör kullanımı yaygınlaşmıştır; doğru frekans seçimi önemlidir.
- Sanal sensörler
  - GPS, kamera, mikrofon, Wi-Fi modülü, Bluetooth modülü sanal sensör olarak değerlendirilebilir.
  - Mesajlar, takvim eventleri de sanal sensör verisi olarak kabul edilebilir (toplantı varsa telefon kendini sessize alabilir).
  - Fiziksel sensörler: kamera, mikrofon, Wi-Fi modülü, Bluetooth modülü, GPS modülü.
- İnsan Activity Recognition API
  - Android'in sunduğu bir API; kişinin hareketlerini takip eder.
  - Hareket değişimi algılandığında arka planda trigger sensörü tetiklenir.
  - Araba, yürüyüş, dans, egzersiz gibi aktiviteleri tanıyabilir.
  - Activity Recognition API kullanılmazsa kendi aktivite tanıma mantığı accelerometer, gyroscope, magnetometer ile yazılabilir.
- Hareketsizlik hesaplama (Su terazisi örneği)
  - Telefon düz bir yüzeye konulduğunda tek bir eksende 9.81, diğer 2 eksende 0 görülür.
  - Eğimli yüzeye konulduğunda değerler eksenlere dağılır.
  - Bu basit kontrol ile yüzeyin eğimli olup olmadığı tespit edilebilir.
  - Daha hassas ölçüm için orientation sensörü (deprecated) yerine gyroscope ve magnetometer kombinasyonu kullanılabilir.
- Jiroskop ve accelerometer farkı
  - Jiroskop: belirli bir eksendeki açısal hızı rad/s cinsinden verir (X, Y, Z etrafında ne kadar hızlı dönüyorsunuz).
  - İvme ölçer: doğrusal ivmeyi ölçer.
  - İvme ölçer yeterli olduğunda jiroskop bilgisine ihtiyaç duyulmayabilir.
  - Düşme veya aktivite hareketlerini tanımada jiroskop kullanılır; yürüme/koşma gibi 3 eksende hareket eden detaylı analizler için jiroskop + manyetometre birlikte kullanılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Sensör seçiminin ve yönetiminin kritikliği
  - Farklı cihazlarda farklı sensörler bulunduğundan, program yazarken bu çeşitliliğin göz önünde bulundurulması gerektiği.
  - Çok sık arka planda değer okunan sensörler için frekans ve enerji tüketiminin dikkatlice yönetilmesi.
- Sensör unregister işleminin pil ömrü için önemi
  - Kullanılmayan sensörlerin mutlaka unregister edilmesi gerektiği; oyun duraklatıldığında sensörün unregister edilmesi örnekleri.
  - Arka planda çalışan servislerde gerekli durumlarda unbind edilmesi gerektiği.
- Sensör veri üretme sırasında uzun iş yapılmaması
  - onSensorChanged, onTrigger gibi metotlarda uzun süren işler yapılmamalı.
  - Bunun yerine farklı thread, go async, job scheduler gibi background taskları kullanılmalı.
- Sanal sensör kavramının genişletilmesi
  - Sensör denilince sadece fiziksel sensörler değil, kamera, mikrofon, GPS, mesajlar, takvim eventleri de düşünülmeli.
  - Veri kaynağı olan her nokta sanal sensör verisi olarak değerlendirilebilir.
- Manifest tanımlamalarının uygulama görünürlüğüne etkisi
  - Google Play Filter ile sensör, kamera gibi parçalar required="true" yapılırsa, o parçaya sahip olmayan telefonlarda uygulama görünmez.
  - required="false" yapılırsa iki farklı yol kodlanmalıdır.
- Android 9 ile gelen sensör kullanımı yaklaşımı
  - Continuous, on-change, one-shot (trigger) reporting mode'larının her birinin uygun kullanım senaryoları.
  - Hangi sensör için hangi modun seçileceğinin bilinmesi gerektiği.

Kısa Tekrar Notları

- Sensör framework: SensorManager, Sensor sınıfı, SensorEvent, SensorEventListener.
- Sensör register/unregister: kullanımda register, kullanım dışı unregister.
- 3 reporting mode: continuous (sürekli), on-change (değişimde), one-shot (trigger).
- Frekans birimi: Hz (saniyede veri sayısı); 50 Hz = 20 ms'de bir.
- Hareket sensörleri: accelerometer, gyroscope, gravity, linear accelerometer.
- Pozisyon sensörleri: magnetometer, proximity.
- Çevresel sensörler: light, temperature, pressure, humidity.
- onSensorChanged: sürekli veri alır; onAccuracyChanged: hassasiyet değişimi.
- 3 eksenli sensör: values[0]=X, values[1]=Y, values[2]=Z.
- Step counter: on-change; accelerometer: continuous; significant motion: one-shot.
- Activity Recognition API: hareket tanıma, trigger sensörü.
- Su terazisi: telefon düz yüzeyde 1 eksen 9.81, diğerleri 0; eğimli yüzeyde dağılır.
- Jiroskop: açısal hız (rad/s); accelerometer: doğrusal ivme.
- Manifest'te required="true" → Google Play'de filtreleme; required="false" → iki yol.

Detaylı Açıklamalar

Dersin başlangıcında hoca, iki haftadır görüşemediklerini, toplamda üç hafta olduğunu, bu sene iki dersin bayramlardan dolayı gerçekleşmediğini belirtmiştir. Dönemin sonuna doğru yaklaştıklarını, üç haftalarının kaldığını söylemiştir. Bugün tempolu bir şekilde ilerleyecekleri, önce sensörlerden (telefonlar üzerinde nasıl veri toplandığı), arkasından broadcast receiver'ı tamamlayacakları, önümüzdeki hafta location based servisleri, arkasından background task'ları, notification'ları, vakit kalırsa mapler ve Android market'e uygulama yükleme prosedürlerini anlatacakları belirtilmiştir. Dönem projesi ile ilgili soru sorulmuş, bonus konusunda Firebase kullanımının ötesinde, bilgilerin lokalde saklanıp telefonun şarja takıldığı anda network'e aktarılmasını kastettiği, Firebase üzerinden de implemente edilebileceği ama özel yapıyı kuranların bonus alacağı söylenmiştir.

Dersin ana konusuna, yani sensörlere geçildiğinde ilk olarak sensör kavramı tanıtılmıştır. Sensörler, cihazların (akıllı telefon, akıllı saat) gözü kulağı olarak kabul edilebilir. Mevcut sensör-

ler insanlardan daha hassas ölçümler yapabilir (düşme tespiti, adım sayma, sıcaklık ölçümü). Cihazlardaki sensörler küçük boyutlarına rağmen yeterli olsa da, profesyonel (plasman) sensörler kadar detaylı ölçüm yapamayabilir. Cihazlar üzerindeki hangi sensörlerin olduğunu bilmek programlama yaparken önemlidir; farklı cihazlarda farklı sensörler bulunabilir. Bu durum program yazarken göz önünde bulundurulmalıdır. Her sensörün yetenekleri farklıdır: ölçebileceği aralık (range), çözünürlük (resolution), ihtiyaç duyduğu güç ve enerji. Bu nedenle özellikle çok sık arka planda değer okunan sensörler için bu özelliklerin gözetilmesi kritik öneme sahiptir.

Sensör türleri detaylı olarak açıklanmıştır. Hareket sensörleri: ivme ölçer (accelerometer), jiroskop (gyroscope), yerçekimi sensörü. Pozisyon sensörleri: manyetometre, proximity (yakınlık) sensörü. Çevresel sensörler: ışık, sıcaklık, basınç, nem sensörü. Donanımsal (hardware) ve yazılımsal (software) sensörler ayrımı vardır; yazılımsal sensörler donanım sensörlerinden elde edilen değerler kullanılarak hesaplanır (örneğin yerçekimi sensörü, doğrusal ivme ölçer, ivme ölçerden yer çekimi çıkarılarak).

Sensör framework'ı dört temel bileşenden oluşur: SensorManager (sensörlere erişim ve sisteme entegrasyon/çıkarma sorumluluğu), Sensor sınıfı (sensörün yeteneklerini paylaşır), SensorEvent (sensörden veri üretildiğinde oluşan obje; verinin ne zaman üretildiği, hareketin değeri, hangi sensörden geldiği bilgilerini saklar), SensorEventListener (tetikleme sırasında değişen değerleri takip etmek için listener).

Sensör kullanım adımları açıklanmıştır. SensorManager objesi oluşturulur (getSystemService ile sensör servisi alınır). getSensorList ile cihazdaki sensörler listelenir (type all veya type accelerometer gibi). Cihazda birden fazla aynı tip sensör olabilir (hastal sebepler, enerji verimliliği, yedeklilik). Sensör özellikleri: getResolution, getMaximumRange, getPower, getVendor, getVersion. Sensör sorgulama: queryIntentActivities benzeri bir kontrol ile ihtiyaç duyulan sensör var mı yok mu tespit edilir. Sensör kaydı: registerListener ile sensör dinlenmeye başlanır, unregister ile durdurulur. Hoca, registerListener'a üç parametre (SensorEventListener, Sensor, frekans) verildiğini, burada Google'ın üreticisi ve versiyon 3 gibi özelliklere göre spesifik sensör seçilebileceğini açıklamıştır.

Sensör frekansı (veri toplama hızı) detaylı olarak ele alınmıştır. Saniyedeki veri toplama hızına "frekans" denir (örneğin 50 Hz = saniyede 50 veri, 20 ms'de bir). Akselerometre ve jiroskop gibi hareket sensörleri saniyede 200 defaya kadar veri toplayabilir. Işık sensörü gibi sensörler daha düşük aralıklarla (saniyede 1-2) değer üretir; sık okuma anlamsızdır. Hangi sensörle çalışılıyorsa çözünürlük ve frekans set edilebilir.

SensorEventListener metotları açıklanmıştır. onAccuracyChanged sensör hassasiyeti değiştiğinde çağrılır. onSensorChanged her veri değişiminde çağrılır; her bir değişim gerçekleştiğinde sensör event üzerinden değerler alınır. Birden fazla sensör okunuyorsa hangi sensörden değer geldiği kontrol edilmelidir. Sürekli veri alan sensörler (accelerometer, gyroscope) "continuous" olarak event üretir. Step counter gibi sensörler sadece değişim olduğunda event üretir ("on-change"). Significant motion sensör gibi trigger sensörler sadece belirli bir olay tetiklendiğinde bir kez event üretir ("one-shot"); event yakalandıktan sonra sensör kendini deaktif eder ve devamlı dinleme rutininde kalmaz.

Sensör veri formatı açıklanmıştır. 3 eksenli sensörler (X, Y, Z) için values[0], values[1], values[2]. 5 eksenli (kalibre olmamış) sensörler için values[0..4]. Tek eksenli sensörler (örn. ışık) için sadece values[0]. Kalibre ve kalibre olmayan sensörlerin farklı veri formatları vardır; kalibre olanlar belirli bir gürültü kontrolü yapılıp değeri ona göre üretir.

Sensör kayıt ve kaldırma (register/unregister) konusu özellikle vurgulanmıştır. Kullanılmayan sensörlerin mutlaka unregister edilmesi gerekir (pil ömrü için). Activity pause olduğunda (örneğin oyun duraklatıldığında) sensör unregister edilebilir. Arka planda çalışan servisler için sensör kaydı gerekli olduğunda, gerekli durumlarda unbind edilerek durdurulabilir. Manifest'te Google Play Filter ile sensör/kamera gibi parçalar required="true" olarak tanımlanabilir; olmayan cihazlarda uygulama Google Play'de görünmez. required="false" yapılırsa iki farklı yol (sensörü olan/olmayan) kodlanabilir.

Sanal sensör kavramı genişletilmiştir. GPS, kamera, mikrofon, Wi-Fi modülü, Bluetooth modülü sanal sensör olarak değerlendirilebilir. Mesajlar, takvim eventleri de sanal sensör verisi olarak kabul edilebilir (toplantı varsa telefon kendini sessize alabilir). Veri kaynağı olan her

nokta sanal sensör verisi olarak değerlendirilebilir. Hoca, “kendi sanal sensörünüzü bu yönde üretme şansınız var” diyerek uygulama özelinde sanal sensör yazılabileceğini belirtmiştir.

İnsan Activity Recognition API açıklanmıştır. Android’in sunduğu bir API; kişinin hareketlerini takip eder. Hareket değişimi algılandığında arka planda trigger sensörü tetiklenir. Araba, yürüyüş, dans, egzersiz gibi aktiviteleri tanıyabilir. Activity Recognition API kullanılmazsa kendi aktivite tanıma mantığı accelerometer, gyroscope, magnetometer ile yazılabilir (Squat hareketi yaparken telefonun üzerinden anlamak kolay değil ama saat üzerinden anlayabilirsiniz).

Hareketsizlik hesaplama (su terazisi örneği) detaylı olarak verilmiştir. Telefon düz bir yüzeye konulduğunda tek bir eksende 9.81, diğer 2 eksende 0 görülür. Eğimli yüzeye konulduğunda değerler eksenlere dağılır. Bu basit kontrol ile yüzeyin eğimli olup olmadığı tespit edilebilir. Daha hassas ölçüm için orientation sensörü (deprecated) yerine gyroscope ve magnetometer kombinasyonu kullanılabilir. Hoca, “orientation sensörü deprikate olmuş bir sensördür, software based” demiştir.

Jiroskop ve accelerometer farkı açıklanmıştır. Jiroskop belirli bir eksendeki açısal hızı rad/s cinsinden verir (X, Y, Z etrafında ne kadar hızlı dönüyorsunuz). İvme ölçer doğrusal ivmeyi ölçer. İvme ölçer yeterli olduğunda jiroskop bilgisine ihtiyaç duyulmayabilir. Düşme veya aktivite hareketlerini tanımada jiroskop kullanılır; yürüme/koşma gibi 3 eksende hareket eden detaylı analizler için jiroskop + manyetometre birlikte kullanılır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.